

SPACE RESCUE SYSTEM

Documentação Técnica e Memorial Descritivo do Protótipo

Componentes da Equipe de Engenharia:

- Artur Henrique Siqueira (RM 566986)
- Davi de Souza Malta (RM 560327)
- Guilherme Cruz Alves (RM 566861)
- Guilherme de Oliveira Scremin (RM 564788)
- Pedro Sales Ferreira (RM 566910)

Instituição: FIAP

Curso: Engenharia de Software

Fase: Challenge Document - Entrega 1, 2 e 3

Data: Junho de 2026

1. Visão Geral do Projeto

O **Space Rescue System** é um ecossistema de missão crítica projetado para atuar no monitoramento emergencial inteligente e coordenação de equipes de busca e salvamento em áreas remotas ou de difícil acesso (como reservas florestais, cadeias de montanhas e ambientes isolados).

O sistema integra conceitos de **Edge Computing** (utilizando dispositivos computacionais de borda baseados em ESP32), comunicação via satélite/rádio simulada por protocolos de rede leves como o **MQTT**, e persistência estruturada em nuvem baseada no padrão **FIWARE / Node-RED**. Este documento descreve as especificações técnicas da aplicação de gerenciamento do terminal do operador rodando em ambiente Python.

Alinhamento com as ODS (ONU): O projeto está diretamente alinhado com o Objetivo de Desenvolvimento Sustentável 15 (Vida Terrestre) ao rastrear e monitorar parâmetros biométricos de fauna silvestre e equipes em áreas remotas, garantindo segurança operacional e controle ecológico.

2. Requisitos e Dependências do Sistema

Para a perfeita execução do terminal do sistema de controle de resgate, certifique-se de cumprir os seguintes requisitos:

2.1. Ambiente de Execução

- **Interpretador:** Python 3.10 ou superior.
- **Sistema Operacional:** Multiplataforma (Compatível com Windows, Linux e macOS através do tratamento nativo de chamadas de limpeza de terminal).

2.2. Dependências e Bibliotecas Nativas

O protótipo foi projetado utilizando estritamente a biblioteca padrão do Python para garantir portabilidade instantânea e conformidade robusta contra quebras de escopo, dispensando gerenciadores de pacotes externos como o `pip`. As bibliotecas chaves são:

- `os`: Utilizada para mapeamento de sistema operacional e comandos internos de limpeza do prompt (`cls` / `clear`).
- `json`: Utilizada para serialização e desserialização de arquivos de banco de dados locais estruturados.
- `random`: Utilizada na simulação de dados dinâmicos de telemetria baseados nos limites de geolocalização.

3. Instruções de Uso e Operação

Siga os passos descritos abaixo para inicializar e utilizar o sistema:

1. Garanta que os arquivos de código fonte (`main.py` e `funcoes.py`) estejam situados no mesmo diretório de trabalho.
2. Execute o terminal na raiz do projeto através do comando: `python main.py`.
3. O sistema invocará automaticamente a função de checagem física `init_files()`, garantindo a criação da pasta interna de persistência `arquivos/` e do banco de dados relacional simulado em JSON `usuarios.json` se estes não existirem.
4. Navegue pelas opções utilizando o teclado numérico conforme as opções do menu interativo.

4. Arquitetura de Software e Critérios de Avaliação

Abaixo detalha-se como as exigências da grade de avaliação (Entrega 2) foram endereçadas na codificação do protótipo funcional enviado:

Critério Avaliado	Pontuação	Evidência de Implementação e Uso Técnico
Manipulação de Listas e Strings	5 Pontos	Uso intensivo de f-strings customizadas para exibição no terminal; varredura e mutação de listas dinâmicas via <code>usuarios.append()</code> e <code>usuarios.remove()</code> .
Conhecimento em Funções	10 Pontos	Modularização em funções puras com tipagem de dados estática (Type Hinting) como <code>init_files() -> None</code> e <code>adicionar_usuario(login: str, senha: str) -> str</code> .
Noções de Estruturas de Dados	10 Pontos	Construção de dicionários aninhados em listas (JSON array) para mapeamento relacional dos dados dos nós sensores do ESP32 (<code>node_id</code> , <code>latitude</code> , <code>batimentos</code>).
Manipulação de Arquivos	10 Pontos	Abertura segura de arquivos usando gerenciadores de contexto (<code>with open(...)</code>) garantindo o fechamento automático de streams I/O em modo leitura (<code>r</code>) e escrita (<code>w</code>).
Tratamento de Exceções	10 Pontos	Blocos de contenção de falhas físicas baseados em <code>try...except (json.JSONDecodeError, IOError, OSError)</code> evitando panes fatais no terminal em falhas de disco.